PROJECT SPECIFICATION

AI Decision Copilot for Customer Retention

Local LLM-Powered Agentic Decision-Support Tool

Abhiram Krishna Varma Kallepalli
Master of Science in Computer and Information Sciences
Saint Louis University | December 2025
Supervisor: Prof. Srikanth Mudigonda
SPS Corps – Analytics Initiative | March 2026

1. Business Problem Statement

Customer churn — the loss of customers to competitors or disengagement — is a persistent, high-cost challenge across subscription-based industries including telecommunications, SaaS, e-commerce, and financial services. Research consistently shows that retaining an existing customer cost significantly less than acquiring a new one, yet most organizations lack tools that help frontline analysts make fast, explainable retention decisions.

The specific decision this project targets is:

> *"Should this customer be flagged as high churn risk, and what action should be taken to retain them?"*

This decision is recurring (made daily by customer success teams), high-impact (directly tied to revenue), and currently under-supported by tools that combine data-driven risk scoring with explainable, actionable recommendations.

Stakeholders

- Customer Success Managers — primary users who monitor account health and renewal likelihood
- Business Analysts — secondary users who analyze churn trends across segments
- Marketing Teams — consumers of recommendations for targeted retention campaigns
- Product Managers — stakeholders who use churn signals to inform product decisions

---

2. Tool Concept and Scope

2.1 Primary Job-To-Be-Done

> *This tool exists to help a business analyst identify high-risk customers and decide the next best retention action by combining rule-based risk scoring with LLM-driven explanation and recommendation.*

2.2 Scope Variant

Selected: Hybrid (Single Record + Summary)

The tool supports two modes:

(1) deep single-customer analysis with full agentic reasoning, and

(2) a lightweight batch summary identifying the top N highest-risk customers. This balances agentic depth with practical business utility while remaining feasible within hardware and timeline constraints.

2.3 Core User Stories

1. As a business analyst, I want to input a customer ID and receive a churn risk classification so that I can quickly identify whether the customer requires intervention.
2. As a business analyst, I want the system to explain the key factors contributing to churn risk so that I can understand the reasoning behind the classification.
3. As a business analyst, I want the tool to recommend a specific retention strategy so that I can take immediate and actionable steps.
4. As a business analyst, I want to view the top N high-risk customers from a dataset so that I can prioritize outreach efforts effectively.
5. As a business analyst, I want consistent and structured outputs for similar inputs so that I can rely on the system in a decision-support context.

2.4 Out-of-Scope

The following are explicitly excluded from this project:

- Real-time data ingestion or integration with live enterprise CRM or data warehouse systems
- Web-based, desktop GUI, or full-stack application interfaces (CLI-based interaction only)
- Training or deploying advanced machine learning models such as logistic regression or neural networks
- Large-scale batch processing with repeated LLM calls across entire datasets without limits
- Multi-agent orchestration or distributed agent architectures
- Retrieval-Augmented Generation (RAG) pipelines with external knowledge bases
- Cloud deployment, cloud inference, or production-grade infrastructure
- Automated email or CRM action execution based on recommendations

---

3. Architecture Design Decisions

3.1 Local LLM Model

Decision	Choice
Model	Phi-3 Mini (3.8B parameters, quantized)
Runtime	Ollama (primary) / LM Studio (fallback)

Decision	Choice
Rationale	Optimized for CPU-only execution on 16GB RAM; ~2–3GB memory footprint; meets 10-second response target
Inference Parameters	temperature = 0 for deterministic outputs; structured prompt templates

3.2 System Design Pattern

Selected: Scripted Pipeline with LLM Checkpoint (Plan-and-Execute hybrid)

The pipeline follows a fixed execution sequence: data ingestion → preprocessing → rule-based scoring → structured summary construction → LLM reasoning → structured output. The LLM operates as a reasoning checkpoint over deterministic tool outputs, not as a free-form generator. This ensures traceability, interpretability, and controlled reasoning flow.

Design Rationale (Deterministic Pipeline + LLM Reasoning Layer)

The system is a deterministic, scripted pipeline that uses a local LLM as a reasoning layer over structured outputs from rule-based components.

- 1 — Data Processing Module: Loads, validates, and preprocesses the CSV into structured customer profiles
- 2 — Rule-Based Risk Scoring Engine: Computes deterministic churn risk scores and extracts contributing factors
- 3 — LLM Reasoning Module (Phi-3 Mini): Receives structured tool outputs and generates grounded explanations and recommendations

The LLM operates only on structured intermediate outputs and does not participate in decision-making. It receives structured intermediate results and reasons over them, ensuring outputs are traceable to specific input features.

3.3 Memory

Selected: Stateless (No Session Memory)

Each query is processed independently. This simplifies implementation, eliminates context overflow risk on limited RAM, and avoids privacy concerns from retaining prior queries. Follow-up queries within a session are not supported in the initial prototype.

3.4 Interaction Style

Selected: Command-Line Interface (CLI)

A CLI interface allows full focus on core analytics and LLM reasoning components without UI development overhead. It also makes the pipeline transparent and easy to evaluate during testing.

4. Functional Requirements

The following statements define the required behaviors of the system. Each statement is independently verifiable.

1. The system shall accept a CSV file containing customer behavioral and demographic data as input.
2. The system shall validate the input dataset against a predefined schema and reject invalid or incomplete files with a clear error message.
3. The system shall parse raw customer records into a structured internal representation for processing.
4. The system shall preprocess missing, null, or inconsistent values using predefined deterministic rules.
5. The system shall compute a churn risk score using a rule-based scoring engine based on selected customer features.
6. The system shall classify each customer into Low, Medium, or High churn risk categories based on computed score thresholds.
7. The system shall extract a structured list of risk-contributing factors for each customer based on rule evaluation results.
8. The system shall generate an intermediate structured summary object containing risk score, category, and contributing factors.
9. The system shall pass the structured summary object to a local LLM (Phi-3 Mini) for reasoning and explanation generation.
10. The system shall generate a natural language explanation of churn risk strictly grounded in structured input features.
11. The system shall generate actionable retention recommendations based on LLM reasoning over structured risk inputs.
12. The system shall expose a CLI interface allowing users to query individual customers and retrieve full risk analysis results.
13. The system shall restrict LLM invocation in batch mode to a maximum of 1,000 customer records per execution. If the number of Medium and High-risk customers exceeds this limit, the system shall prioritize selection based on descending churn risk score and select the top 1,000 highest-risk customers for LLM-based explanation generation.
14. The system shall generate a --summary output that includes aggregated statistics of the dataset, including total customers processed, count of Low/Medium/High risk classifications, and average MonthlyCharges per risk category.

---

5. Non-Functional Requirements

5.1 Response Latency

The system shall generate a complete response (risk score, explanation, and recommendation) within 10 seconds per individual customer query on a CPU-based local environment with no dedicated GPU.

5.2 Offline Operation

The system shall operate fully offline after model initialization and shall not require internet connectivity during execution. All inference shall run locally using Ollama or LM Studio.

5.3 Privacy

- The system shall process all customer data locally on the user's machine.
- The system shall not transmit any customer data to external APIs or services.
- The system shall not persist raw customer data unless explicitly instructed by the user.

5.4 Reproducibility

The system shall produce identical risk classifications for identical inputs. Rule-based scoring is fully deterministic (100% reproducible). LLM-generated explanations are expected to remain highly consistent under fixed parameters (same prompt structure and temperature = 0), with minor variability possible due to runtime or backend-level non-determinism. Structured outputs (risk score, category, and contributing factors) shall remain consistent.

5.5 Failure Modes

- The system shall validate input schema and reject files with missing required fields with a descriptive error message.
- The system shall handle missing or corrupted values using fallback imputation strategies without crashing.
- The system shall return a controlled fallback response when the LLM fails or produces invalid output after two retry attempts.
- The system shall handle an unknown customer ID gracefully by returning: "Customer ID [X] not found in dataset."

---

6. Interface Requirements

The system shall provide a command-line interface (CLI) supporting the following operations:

6.1 Single Customer Analysis

```
python main.py --file customers.csv --customer_id 101
```

Returns full churn risk analysis for a specific customer including risk score, risk category, contributing factors, LLM explanation, and retention recommendation.

6.2 Batch Processing

```
python main.py --file customers.csv --all
```

Processes all customers, applies rule-based scoring to all records, and invokes the LLM only for Medium and High risk customers (capped at 1,000 records). Generates a complete risk classification report.

6.3 Summary Statistics

```
python main.py --file customers.csv --summary
```

Outputs aggregated statistics: total customers processed, count of Low/Medium/High risk classifications, and average MonthlyCharges per risk category. No LLM invocation required.

6.4 Example Output (Single Customer)

```
Customer ID: 7590-VHVEG
--------------------------------------------
Risk Score:    8 / 10
Risk Level:    HIGH
--------------------------------------------
Key Risk Factors:
- Month-to-month contract
- Fiber optic internet with no security services
- Short tenure (2 months)
- High monthly charges ($85.70)

Explanation:
The customer exhibits multiple high-risk behavioral indicators
including lack of long-term commitment and high cost sensitivity.

Recommendation:
Suggest conversion to annual contract. Encourage enabling
OnlineSecurity and TechSupport. Flag for retention offer.
```

---

7. Data Requirements

7.1 Dataset

The system shall use the IBM Telco Customer Churn dataset sourced from Kaggle:

https://www.kaggle.com/datasets/blastchar/telco-customer-churn

The dataset contains 7,043 customer records with 21 attributes covering identity, demographics, account information, service usage, and churn status.

7.2 Key Features Used

Feature	Type	Role in Scoring
tenure	Numeric	Low tenure = higher risk
Contract	Categorical	Month-to-month = high risk
MonthlyCharges	Numeric	High charges = higher risk
TechSupport	Categorical	No support = higher risk
OnlineSecurity	Categorical	No security = higher risk
InternetService	Categorical	Fiber optic = risk signal
PaymentMethod	Categorical	Electronic check = risk signal
Churn	Binary	Ground truth for evaluation

7.3 Preprocessing Steps

- TotalCharges: Convert blank strings to numeric values; impute missing values with the column median.
- Contract: Map to risk scoring weights (Month-to-month = 2, One year = 1, Two year = 0), where higher values represent higher churn risk rather than a true ordinal relationship.
- SeniorCitizen: Already binary (0/1); no transformation required.
- Categorical **yes/no** fields: Standardize to lowercase for consistent rule evaluation.
- customerID: Retain as string index; exclude from scoring computation.

7.4 Local Storage Policy

- The dataset shall be stored locally in CSV format in the /data directory.
- No external database systems or cloud storage shall be used.
- Session logs shall be written to /outputs/session.log and retained locally.

8. Evaluation Plan

8.1 Quantitative Metrics

Metric	Target	Measurement Method
Risk Classification Accuracy	≥70% agreement with Churn ground truth (used as a validation benchmark rather than a trained model accuracy metric)	Compare High-risk classifications to Churn=Yes labels
Response Latency	< 10 seconds per query	System timer per query execution
Reproducibility Rate	≥90% identical classifications	Run same input 10 times; count consistent outputs
LLM Schema Compliance	100% valid JSON structure	JSON schema validator on all LLM outputs

8.2 LLM Output Evaluation Rubric

Each LLM output is scored on a 0–3 scale by a human reviewer:

Score	Criteria
3	Fully correct: ≥3 correct risk factors cited + valid recommendation + no hallucinated reasoning
2	Correct factors identified but explanation alignment is weak or recommendation is partially generic
1	Partial factor coverage (<3 correct factors) or weak mapping between factors and recommendation
0	Incorrect or missing risk factors, or hallucinated reasoning not grounded in input features

Target: Average LLM alignment score ≥ 2.0 across all test cases. At least 8 of 10 test cases must pass.

8.3 Test Scenarios

Scenario 1 — High Risk Customer (Fully Specified)

Input:

Customer Record: customerID = 7590-VHVEG, tenure = 2, Contract = Month-to-month, MonthlyCharges = 85.70, TotalCharges = 171.40, InternetService = Fiber optic, OnlineSecurity = No, TechSupport = No, PaymentMethod = Electronic check.

Expected Output:

Risk Category = High, Risk Score ≥ 7, at least 3 contributing factors explicitly identified (Month-to-month contract, Fiber optic with no security, Short tenure), Recommendation includes contract conversion and security service enablement.

Scenario 2 — Low Risk Customer

Input:

tenure ≥ 48 months, Contract = Two year, MonthlyCharges < 50, OnlineSecurity = Yes, TechSupport = Yes.

Expected Output:

Risk Category = Low, Risk Score ≤ 3, minimal intervention recommendation.

Scenario 3 — Edge Case: Missing Data

Input:

CSV with TotalCharges field blank, OnlineSecurity field null for selected customer.

Expected Output:

System applies fallback imputation, generates valid output without crashing, logs a warning to session.log.

Scenario 4 — Unknown Customer ID

Input:

--customer_id 99999 where ID does not exist in dataset.

Expected Output:

System returns controlled message “Customer ID 99999 not found in dataset.” without crashing.

Scenario 5 — Invalid CSV Schema

Input:

CSV missing the Contract column.

Expected Output:

System rejects file with error message specifying missing required field. No partial processing attempted.

Scenario 6 — Corrupted Numeric Field

Input:
MonthlyCharges = "N/A" or non-numeric value in dataset

Expected Output:

System handles invalid numeric conversion using fallback imputation strategy (e.g., median or predefined default), logs a warning, and continues processing without crashing.

---

9. Nine-Week Implementation Timeline

Week	Focus	Deliverables	Definition of Done
1	Problem Finalization + Evaluation Design	Finalized spec on GitHub, EDA notebook, 10 test cases, repo structure	Spec committed; EDA shows Churn distribution; test cases cover all risk levels
2	Architecture + Tech Stack	Ollama + Phi-3 Mini installed, all modules scaffolded, requirements.txt	LLM responds locally in <10s; all modules have function signatures
3	Vertical Slice Part 1 (Data Pipeline)	data_loader.py, preprocessor.py, scorer.py functional; 5 unit tests	customerID 7590-VHVEG scores as High Risk; all unit tests pass
4	Vertical Slice Part 2 (LLM Integration)	llm_interface.py connected; full end-to-end pipeline for one customer	Full output for 7590-VHVEG matches expected test scenario in <10s
5	Batch Mode + Summary Command	--all and --summary CLI flags functional	--summary matches spec format; --all processes 100 records without error
6	Guardrails + Logging + Robustness	Input validation, fallback responses, session.log output	System handles all 5 edge cases without crashing; logs generated
7	Evaluation Execution	All 10 test cases scored; evaluation report committed	8/10 test cases pass; avg LLM alignment score ≥ 2.0
8	Polish + Demo Script	README.md, demo script, bug fixes from Week 7	New user can run system from README alone; demo runs without errors
9	Final Report + Postmortem	Final report (2,000+ words), complete GitHub repo, email to professor	Repo is public, organized, and complete with all deliverables

10. Skills and Technology Learning Map

Weeks 1–2 — Foundation Skills

Skill	Tool / Library	Practice Task
CSV loading and filtering	pandas	Load Telco CSV; print churn count and avg MonthlyCharges for month-to-month contracts
JSON parsing	json (stdlib)	Convert risk result dict to JSON string; parse back; print risk_category field
CLI building	argparse	Build script accepting --file and --customer_id; print confirmation message
Dependency management	venv, pip	Create clean environment; install all dependencies; export requirements.txt
Version control	Git, GitHub	Create repo with /data /src /tests /docs /outputs; commit initial scaffold

Weeks 3–4 — Core Implementation Skills

Skill	Tool / Library	Practice Task
Rule-based scoring logic	Python if/else, pandas	Write score_customer() returning score, category, factors; test on 7590-VHVEG
Structured prompt engineering	Ollama API	Engineer prompts to enforce structured JSON outputs from LLM; implement validation, retry mechanisms, and parsing strategies to handle occasional malformed or non-JSON responses
JSON schema validation	json, re	Validate LLM output has required fields; implement retry on failure
LLM API integration	ollama Python library	Run Phi-3 Mini locally; confirm response under 10 seconds
String parsing for LLM output	str methods, re	Extract JSON from LLM response that includes preamble text

Weeks 5–6 — Integration and Robustness Skills

Skill	Tool / Library	Practice Task
Batch processing	pandas itertuples, tqdm	Process 50 records; show progress bar; filter to Medium/High only
Structured logging	logging (stdlib)	Log timestamp, customer_id, risk_category, response_time, llm_valid per query
Exception handling	try/except/finally	Handle missing CSV column, unknown ID, and all-null row without crashing
CLI output formatting	Python string formatting	Print aligned risk report table to terminal

Weeks 7–9 — Evaluation and Reporting Skills

Skill	Tool / Library	Practice Task
Unit testing	pytest	Write test_high_risk(), test_low_risk(), test_missing_field(); confirm all pass
Structured evaluation	Manual rubric (0–3 scale)	Score 5 LLM outputs; record in evaluation table; identify prompt improvements
Technical writing	Markdown	Write README with setup instructions usable by a new user
Final reporting	Markdown / Word	Document design decisions, results, limitations, and lessons learned (2,000+ words)